

P1119R0

ABI for

std::hardware_{constructive,destructive}_interference_size

Published Proposal, 22 June 2018

This version:

<http://wg21.link/d1119r0>

Authors:

[JF Bastien](#) (Apple)

[Olivier Giroux](#) (NVIDIA)

[Jonathan Wakely](#) (RedHat)

[Hal Finkel](#) (Argonne National Laboratory)

[Thomas Rodgers](#) (RedHat)

[Matthias Kretz](#) (GSI)

Audience:

SG1, LEWG, LWG

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Source:

github.com/jfbastien/papers/blob/master/source/p1119r0.bs

Abstract

std::hardware_{constructive,destructive}_interference_size exposes potential ABI issues, and that's OK. This position paper clarifies the committee's position.

Table of Contents

1 Wording

2 Discussions

3 Pushback

4 Polls

References

Informative References

§ 1. Wording

[P0154R1] introduced `constexpr` `std::hardware_{constructive,destructive}_interference_size` to C++17:

Header `<new>` synopsis [**new.syn**]:

```
namespace std {  
    // ...  
    // 21.6.5, hardware interference size  
    inline constexpr size_t hardware_destructive_interference_size = imple  
    inline constexpr size_t hardware_constructive_interference_size = impl  
    // ...  
}
```

Hardware interference size [**hardware.interference**]:

```
inline constexpr size_t hardware_destructive_interference_size = impleme
```

This number is the minimum recommended offset between two concurrently-accessed objects to avoid additional performance degradation due to contention introduced by the implementation. It shall be at least `alignof(max_align_t)`.

[*Example:*

```
struct keep_apart {  
    alignas(hardware_destructive_interference_size) atomic<int> cat;  
    alignas(hardware_destructive_interference_size) atomic<int> dog;  
};
```

— *end example*]

```
inline constexpr size_t hardware_constructive_interference_size = implem
```

This number is the maximum recommended size of contiguous memory occupied by two objects accessed with temporal locality by concurrent threads. It shall be at least `alignof(max_align_t)`.

[*Example:*

```
struct together {  
    atomic<int> dog;  
    int puppy;  
};  
struct kennel {  
    // Other data members...  
    alignas(sizeof(together)) together pack;  
    // Other data members...  
};  
static_assert(sizeof(together) <= hardware_constructive_interference_siz
```

— *end example*]

§ 2. Discussions

The paper was discussed in:

- [SG1 Kona](#)
- [LEWG Kona](#)
- [LEWG Jacksonville](#)
- [LWG Jacksonville](#)

ABI issues were considered in these discussions, and the committee decided that having these values was worth the potential pain points. ABI issues can arise as follows:

1. A developer asks the compiler to generate code for multiple targets of the same ISA, and these targets prefer different interference sizes.
2. A developer indicates that code should be generated for heterogeneous system (such as CPU and GPU), which prefer different interference sizes.
3. A developer uses different compilers, and links the result together.

A further ABI issue was added by [\[P0607r0\]](#) by making the variables `inline`: in case 1. above the interference size values differ between translation units, which is a problem if they are used in an ODR-relevant context. That paper noted:

[Drafting notes: The removal of the explicit `static` specifier for the namespace-scope constants `hardware_destructive_interference_size` and `hardware_constructive_interference_size` is still required because adding `inline` alone would still not solve the ODR violation problem here. — end drafting notes]

This change indeed fixes the ODR issue where two translation units translated with the same interference size values may violate ODR when used with e.g. `std::max`. It however introduces a new ODR issue for case 1. above.

Richard Smith and Tim Song propose changing the definition to:

```
static constexpr const std::size_t& hardware_destructive_interference_size  
static constexpr const std::size_t& hardware_constructive_interference_size
```

We propose a discussion and poll on this topic.

§ 3. Pushback

The maintainers of clang and GCC have [discussed an implementation strategy](#), but received pushback based on the above ABI issues. The messaging from the committee wasn't clear that ABI issues were discussed and the proposal accepted despite these issues. This type of ABI problem is difficult or impossible to warn about, some implementors are worried.

Some implementors are worried that they have the following choices when implementing, and are unsure which approach to take:

1. Pick a value once for each ABI and cast it in stone forever, even if microarchitectural revisions cause the values to change.
2. Change the value between microarchitectures, even though that's an ABI break?
3. Something else.

The authors believe that the ABI issues are acceptable because:

- As demonstrated in the original paper, developers already write code like this, using macros. Any ABI issue that exist with this proposal already existed before the proposal.
- Many uses of these values have no ABI breakage potential because they only target one variant of one ISA.
- The usecase for these values is to lay out datastructures. These datastructures shouldn't be shared across translation units which follow different ABIs.
- Similar ABI issues already exist with `max_align_t` and `intmax_t`.
- Implementations can offer compiler flags which specifically control ABI. For example, `-mcpu` could keep the ABI stable, but `-mcpu-abi` would change it.

§ 4. Polls

We propose the following poll for SG1:

The committee understands the ABI issues with `std::hardware_{constructive,destructive}_interference_size`, yet chooses to standardize these values nonetheless.

The committee could also consider adding a note to point out ABI issues with these values. This would be a novel note, since ABI isn't discussed in the Standard.

We propose the following poll for SG1, LEWG, and LWG:

Both ODR issues should be addressed, the type should therefore be changed to `static constexpr const std::size_t&`.

Not all authors of this paper are in favor of this direction, but all agree the discussion is worth having.

§ References

§ Informative References

[P0154R1]

JF Bastien, Olivier Giroux. [constexpr std::thread::hardware_{true,false}_sharing_size](https://wg21.link/p0154r1). 3 March 2016. URL: <https://wg21.link/p0154r1>

[P0607r0]

Daniel Krugler. [Inline Variables for the Standard Library](https://wg21.link/p0607r0). 27 February 2017. URL: <https://wg21.link/p0607r0>